Lesson 4 Worksheet: The Manhattan Algorithm — ANSWER KEY

Part 1: Manhattan Distance

Calculate the Manhattan distance for each pair. Remember: Manhattan distance = |row difference| + |column difference|.

Start	Destination	Row Distance	Col Distance	Manhattan Distance
(0, 0)	(2, 3)	|2 - 0| = 2	|3 - 0| = 3	5
(3, 3)	(0, 0)	|0 - 3| = 3	|0 - 3| = 3	6
(1, 0)	(1, 3)	|1 - 1| = 0	|3 - 0| = 3	3
(0, 2)	(3, 2)	|3 - 0| = 3	|2 - 2| = 0	3
(2, 1)	(2, 1)	|2 - 2| = 0	|1 - 1| = 0	0

Which pair requires the most steps? (3, 3) to (0, 0) with a Manhattan distance of 6.

Which pair requires zero steps? Why? (2, 1) to (2, 1) — the start and destination are the same position, so no movement is needed.

Part 2: Determine the Step Values

Start	Destination	row_step	col_step
(0, 0)	(3, 2)	+1	+1
(3, 3)	(0, 1)	-1	-1
(1, 3)	(3, 0)	+1	-1
(2, 0)	(0, 3)	-1	+1

Start	Destination	row_step	col_step
(0, 0)	(0, 3)	-1 (but doesn't matter)	+1

For the last row, the start row and destination row are the same. Does the value of row_step matter? Why or why not?

No, the value of row_step does not matter. Because the start row equals the destination row, the row `while` loop condition is already false, so the loop never executes. The value of row_step is never used.

Part 3: Hand-Trace the Algorithm

Path 1: (0, 0) to (3, 2)

- row_step = +1, col_step = +1

Row moves:

Step	Action	Position
Start	—	(0, 0)
1	Row +1	(1, 0)
2	Row +1	(2, 0)
3	Row +1	(3, 0)

Column moves:

Step	Action	Position
4	Col +1	(3, 1)
5	Col +1	(3, 2)

Complete path: (0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2)

Total steps: 5 **Manhattan distance:** 5 **Do they match?** YES

Path 2: (3, 3) to (0, 1)

- row_step = -1, col_step = -1

Row moves:

Step	Action	Position
Start	—	(3, 3)
1	Row -1	(2, 3)
2	Row -1	(1, 3)
3	Row -1	(0, 3)

Column moves:

Step	Action	Position
4	Col -1	(0, 2)
5	Col -1	(0, 1)

Complete path: (3,3) → (2,3) → (1,3) → (0,3) → (0,2) → (0,1)

Total steps: 5 **Manhattan distance:** 5 **Do they match?** YES

Path 3: (1, 3) to (3, 0)

- row_step = +1, col_step = -1

Row moves:

Step	Action	Position
Start	—	(1, 3)
1	Row +1	(2, 3)
2	Row +1	(3, 3)

Column moves:

Step	Action	Position
3	Col -1	(3, 2)
4	Col -1	(3, 1)

Step	Action	Position
5	Col -1	(3, 0)

Complete path: (1,3) → (2,3) → (3,3) → (3,2) → (3,1) → (3,0)

Total steps: 5 **Manhattan distance:** 5 **Do they match?** YES

Path 4: (2, 2) to (2, 0) (same row!)

- row_step = doesn't matter, col_step = -1

Does the row loop execute? NO **Why?** The start row (2) already equals the destination row (2), so the while loop condition is false and the loop never executes.

Column moves:

Step	Action	Position
Start	—	(2, 2)
1	Col -1	(2, 1)
2	Col -1	(2, 0)

Complete path: (2,2) → (2,1) → (2,0)

Total steps: 2 **Manhattan distance:** 2 **Do they match?** YES

Path 5: (0, 1) to (3, 1) (same column!)

- row_step = +1, col_step = doesn't matter

Does the column loop execute? NO **Why?** The start column (1) already equals the destination column (1), so the while loop condition is false and the loop never executes.

Row moves:

Step	Action	Position
Start	—	(0, 1)
1	Row +1	(1, 1)

Step	Action	Position
2	Row +1	(2, 1)
3	Row +1	(3, 1)

Complete path: (0,1) → (1,1) → (2,1) → (3,1)

Total steps: 3 **Manhattan distance:** 3 **Do they match?** YES

Part 4: Draw the Paths on the Grid

Most paths make an **L-shape** (right angle turn where the algorithm switches from row moves to column moves).

```
          Col 0        Col 1        Col 2        Col 3
            |            |            |            |
Row 0 ---[0,0]———————[0,1]———————[0,2]———————[0,3]---
            | 1↓         5↓                      2←|
Row 1 ---[1,0]———————[1,1]———————[1,2]———————[1,3]---
            | 1↓         5↓                      2↑ 3↓
Row 2 ---[2,0]———————[2,1]———————[2,2]———————[2,3]---
            | 1↓    4←   |      4←   |          2↑ 3↓
Row 3 ---[3,0]———————[3,1]———————[3,2]———————[3,3]---
             1→→→→→→→→→→→→→→→→→→→→→1    3←←←←←3
                                         2←←←←←2
```

What shape do most of the paths make? An L-shape (right angle). The algorithm moves along rows first, then turns and moves along columns.

Which paths are straight lines instead of L-shapes? Paths 4 and 5. Path 4 moves only along columns (same row), and Path 5 moves only along rows (same column).

Part 5: Verify Your Understanding

1. **The Manhattan algorithm always moves *rows* first, then *columns*.**
2. **If the destination row is SMALLER than the start row, the robot moves *up*.**
3. **If the destination column is LARGER than the start column, the robot moves *right*.**

4. When the start and destination are in the same row, the row loop *does not execute.*
5. When the start and destination are the same position, the path contains *1* position(s).
6. The number of steps in a path always equals the *Manhattan distance.*

Reflection

You traced 5 different paths by hand today. In the next lesson, you will write a Python class that does this automatically. Why is it important to trace the algorithm by hand BEFORE writing code?

Tracing by hand helps you understand the algorithm's logic step by step, so you know exactly what your code should do. It lets you catch mistakes in your thinking before they become bugs. It also gives you known correct outputs that you can use to verify your code produces the right results. If your code gives a different answer than your hand trace, you know something is wrong and can debug it.

Next Lesson: We'll implement the Manhattan algorithm as a Python class with a `compute_path()` method!